UNIVERSIDAD TECNOLÓGICA DE PEREIRA

INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

HPC: HIGH PERFORMANCE COMPUTING

Análisis de Rendimiento de Multiplicación de Matrices: Secuencial vs Paralela

Autor: Jorge Luis López Grajales

Asignatura: HPC: High Performance Computing

Docente: Ramiro Andrés Barrios Valencia

Fecha: 19 de marzo de 2026

Pereira – Colombia

ÍNDICE

I. Objetivo 2

II. Marco Conceptual 2

II-A. High Performance Computing (HPC) 2

II-B. Multiplicación Matricial . 2

II-C. Programación Paralela e Hilos (Threads) 2

II-D. Speedup 2

III. Marco Contextual 3

III-A. Especificaciones del Entorno 3

IV. Desarrollo e Implementación 3

IV-A. Implementación de Hilos (Pthreads) 3

IV-B. Implementación de Procesos (Shared Memory) . 3

IV-C. Optimización de Caché (Localidad de Datos) . . 4

IV-D. Optimización por Compilador (`-Ofast`) 4

IV-E. Optimización de hilos con "memoria compartida 4

V. Tablas por archivo 4

V-A. 2hilos.doc 5

V-B. 2procesos.doc 5

V-C. 4hilos.doc 6

V-D. 4procesos.doc 6

V-E. 6hilos.doc 7

V-F. 6procesos.doc 7

V-G. 8hilos.doc 8

V-H. 8procesos.doc 8

V-I. cache.doc 9

V-J. ofast.doc 9

V-K. standar.doc 10

V-L. hilos8.2.doc 10

VI. Gráficas 10

VII. Conclusiones 12

Referencias 13

I. Objetivo

Este informe presenta el análisis de tiempo de ejecución y speedup para las variantes de multiplicación de matrices incluidas en el archivo de datos `resultados.csv`. Las gráficas se incluyen como evidencia visual del comportamiento observado.

II. Marco Conceptual

II-A. High Performance Computing (HPC)

La computación de alto rendimiento se refiere al uso de sistemas de cómputo potentes para resolver problemas complejos que requieren una gran capacidad de procesamiento y manejo de datos. En este contexto, se busca optimizar el uso del hardware disponible para reducir los tiempos de ejecución.

II-B. Multiplicación Matricial

Es una operación fundamental en el álgebra lineal. Dados dos matrices A de orden $m \times n$ y B de orden $n \times p$, el resultado es una matriz C de orden $m \times p$. Su complejidad algorítmica secuencial estándar es $O(n^3)$, lo que la hace una candidata ideal para la paralelización.

II-C. Programación Paralela e Hilos (Threads)

La programación paralela permite la ejecución simultánea de múltiples tareas para mejorar la eficiencia. Los hilos son unidades básicas de utilización de la CPU que comparten el mismo espacio de direcciones de memoria, facilitando la comunicación en algoritmos de división y conquista como la multiplicación de matrices.

II-D. Speedup

Es la métrica que mide la aceleración obtenida al ejecutar un programa en paralelo en comparación con su versión secuencial. Se define mediante la fórmula:

$$S_p = \frac{T_1}{T_p}$$

Donde T_1 es el tiempo secuencial y T_p es el tiempo con p procesadores o hilos.

III. Marco Contextual

III-A. *Especificaciones del Entorno*

Para garantizar la fidelidad de los datos, las muestras se tomaron durante varias sesiones nocturnas, ejecutando un script automatizado en segundo plano para minimizar la interferencia de otros procesos del sistema.

- **Sistema Operativo:** Arch Linux x86_64 (Kernel 6.19.8-arch1-1).
- **Hardware:** ASUS TUF Gaming F15 FX506LH.
- **Procesador (CPU):** Intel(R) Core(TM) i5-10300H (8 hilos lógicos) @ 4.50 GHz.
- **Memoria RAM:** 16 GB DDR4.
- **GPU:** NVIDIA GeForce GTX 1650 Mobile / Max-Q.
- **Terminal:** kitty 0.46.0 con Shell fish 4.5.0.
- **Metodología:** Automatización mediante scripts y lenguaje C para las pruebas de rendimiento.

IV. Desarrollo e Implementación

En esta sección se detallan las cuatro estrategias implementadas para abordar la multiplicación de matrices, todas las implementaciones miden el `walltime` de la misma manera con la libreria `time.h`.

IV-A. *Implementación de Hilos (Pthreads)*

Se desarrolló una solución utilizando la librería `pthread.h`. En esta versión, el espacio de trabajo se divide por filas, asignando a cada hilo un rango específico de la matriz resultante.

- **Gestión de Memoria:** Se utilizó asignación dinámica mediante `malloc` para cada fila de forma independiente.
- **Limitación:** Esta estructura genera una dispersión de los datos en la RAM, lo que incrementa los fallos de caché (*cache misses*) al no garantizar que las filas sean contiguas físicamente.

IV-B. *Implementación de Procesos (Shared Memory)*

A diferencia de los hilos, esta versión utiliza la técnica de comunicación entre procesos (IPC).

- **Memoria Contigua:** Se reservó un único bloque de memoria compartida para toda la matriz usando `shmget` y se mapeó al espacio de direcciones de cada proceso con `shmat`.

- **Ventaja:** Al ser un bloque de memoria lineal y contiguo, el procesador puede predecir los siguientes datos a leer (*Hardware Prefetching*), lo que explica por qué esta versión superó en rendimiento a la de hilos.

IV-C. Optimización de Caché (Localidad de Datos)

Se implementó una versión específica para mejorar la localidad temporal y espacial. En lugar de acceder a la matriz B por columnas, se buscó optimizar el recorrido de los bucles para asegurar que los datos cargados en el caché L1 y L2 del i5-10300H se reutilizaran la mayor cantidad de veces posible antes de ser descartados.

IV-D. Optimización por Compilador (`-Ofast`)

La version secuencial fué compilada utilizando el compilador GCC con la bandera de optimización agresiva `-Ofast`.

- **Vectorización:** `-Ofast` permite el uso de instrucciones SIMD (Single Instruction, Multiple Data), lo que permite que el procesador procese múltiples elementos de la matriz en un solo ciclo de reloj.
- **Aritmética de punto flotante:** A diferencia de `-O3`, `-Ofast` relaja algunas restricciones de precisión para ganar velocidad extrema.

IV-E. Optimización de hilos con "memoria compartida"

Esta versión se diferencia de las demás ejecuciones de hilos en que la matriz no se reserva fila por fila; en su lugar, se realiza un único `malloc` para todo el bloque de datos. Al evitar que la matriz se divida en múltiples segmentos dispersos en la memoria RAM, se garantiza la **contigüidad física** de los datos.

Esta técnica reduce drásticamente el *overhead* de gestión del montículo (*heap*) y optimiza la jerarquía de caché del procesador. Al estar los datos alineados linealmente, el *hardware prefetcher* de la CPU puede predecir con mayor exactitud los siguientes elementos necesarios, minimizando los fallos de caché (*cache misses*). Como resultado, los hilos pasan menos tiempo esperando la llegada de datos desde la memoria principal y más tiempo realizando cómputo útil, acercando el rendimiento de la implementación de hilos al de la versión de procesos con memoria compartida.

V. TABLAS POR ARCHIVO

Cada tabla contiene métricas descriptivas por tamaño: número de ejecuciones (n), promedio, desviación estándar, mínimo y máximo.

V-A. *2hilos.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	2.0067	0.1372	1.8619	2.2773
1500	10	8.2282	0.3052	7.7406	8.9302
2000	10	27.6126	5.8263	24.9230	43.8947
2500	10	47.6105	2.9198	45.2708	53.4145
3000	10	93.8420	5.1064	89.9402	103.2493
3500	10	131.6773	9.6308	123.8815	155.7349
4000	10	223.4914	9.6977	215.4968	242.5050
4500	10	285.8682	19.8710	247.1849	320.4024
5000	9	438.0608	21.1774	403.8498	476.8893

V-B. *2procesos.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	11	1.4559	0.1033	1.3469	1.7531
1500	11	6.6703	0.1347	6.4367	6.9590
2000	11	12.6276	0.2426	12.2598	13.0176
2500	10	35.7557	0.3023	35.4366	36.2444
3000	10	52.1475	0.8038	50.4536	53.0910
3500	10	102.7261	0.4655	102.0789	103.2441
4000	10	118.1111	0.7487	117.0998	119.0864
4500	10	221.2858	1.3806	218.5130	222.8925
5000	10	257.9117	1.1981	256.2736	259.1862

V-C. *4hilos.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	0.8984	0.0759	0.8561	1.1122
1500	10	3.9097	0.0947	3.8473	4.1681
2000	10	12.4625	0.1666	12.3303	12.9187
2500	10	23.2858	0.3444	22.4386	23.5493
3000	10	46.5390	1.1852	43.2088	47.1641
3500	10	68.7749	0.3013	68.3548	69.3226
4000	10	114.7310	2.7405	107.0290	116.3428
4500	10	153.2144	5.0087	139.3153	156.1987
5000	10	231.1724	6.1117	213.9494	234.6759

V-D. *4procesos.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	0.7589	0.0212	0.7398	0.7871
1500	10	3.2310	0.0423	3.1895	3.3179
2000	10	6.5115	0.0759	6.4168	6.6486
2500	10	18.7832	0.1157	18.6617	19.0138
3000	10	28.4064	0.1901	28.1863	28.7508
3500	10	55.5041	0.2353	55.1818	55.9214
4000	10	67.0107	0.2726	66.7139	67.5795
4500	10	120.9909	0.2696	120.5858	121.3468
5000	10	144.4435	0.3495	143.9131	145.0448

V-E. *6hilos.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	1.1180	0.0634	1.0642	1.2903
1500	10	4.3264	0.0985	4.1879	4.5538
2000	10	12.4278	0.1413	12.2374	12.6093
2500	10	23.5262	0.5181	22.7694	24.7445
3000	10	46.1585	1.0038	45.3030	48.1579
3500	10	69.4506	7.5979	66.0443	90.8402
4000	10	110.9757	4.6381	104.2047	120.7234
4500	10	139.9930	7.6836	130.5658	150.5428
5000	9	205.5319	16.0815	191.0591	229.6631

V-F. *6procesos.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	0.8866	0.0567	0.8142	0.9778
1500	10	3.1225	0.0985	3.0292	3.3100
2000	10	6.9593	0.1211	6.8174	7.2135
2500	10	17.0884	0.2036	16.8832	17.3996
3000	10	27.5571	0.1907	27.3014	27.9322
3500	10	49.3295	0.3423	48.8543	49.9255
4000	10	65.8690	0.5421	64.8868	66.8123
4500	10	107.1653	3.2646	104.7159	116.1272
5000	10	135.8398	0.8316	134.5018	137.5353

V-G. 8hilos.doc

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	1.1160	0.0410	1.0638	1.1782
1500	10	4.3505	0.0557	4.2753	4.4436
2000	10	12.1854	0.1992	11.7064	12.4264
2500	10	21.9544	0.4183	20.8161	22.3811
3000	10	43.6016	1.7743	38.5685	44.4233
3500	10	61.8638	1.5336	57.6504	63.2404
4000	9	105.6425	5.5187	91.5884	108.8921
4500	9	135.7548	9.6078	122.0963	143.0793
5000	9	201.9709	20.6883	174.1196	217.3996

V-H. 8procesos.doc

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	0.8206	0.0137	0.7978	0.8343
1500	10	2.8855	0.0245	2.8431	2.9225
2000	10	7.0192	0.0396	6.9594	7.0910
2500	10	14.8337	0.0786	14.6886	14.9567
3000	10	25.9498	0.1093	25.6962	26.0694
3500	10	42.4023	0.3106	42.0464	42.9584
4000	10	63.6251	0.6956	63.0044	65.2234
4500	10	90.7738	1.1285	89.6858	93.0762
5000	10	124.4605	1.5064	122.9131	128.1742

V-I. *cache.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	10	3.5308	0.1211	3.3868	3.8070
1500	10	11.8760	0.4484	11.4053	12.4735
2000	10	27.0297	0.8656	25.7134	28.3827
2500	10	52.4284	1.2314	50.3931	54.1059
3000	10	89.8361	1.4070	87.4441	92.1417
3500	10	138.1847	1.6680	136.7814	142.3033
4000	10	207.9656	3.7357	203.9627	213.9142
4500	10	290.7592	4.0385	285.8013	297.9978
5000	10	396.8677	6.7854	390.8769	408.2865

V-J. *ofast.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	14	0.8418	0.0545	0.7226	0.9106
1500	14	4.8906	0.2986	4.5851	5.7424
2000	14	25.4283	2.7857	16.5126	29.4348
2500	14	52.3347	5.9381	32.0395	57.5148
3000	14	95.6798	11.5321	55.6549	99.3562
3500	14	157.8343	23.9672	74.6534	165.7760
4000	14	221.3891	49.8769	125.3522	252.5623
4500	13	340.0932	70.7591	172.7757	374.1270
5000	13	494.9240	76.9691	253.9533	529.1496

V-K. *standar.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	12	4.7233	0.7069	4.2351	6.7187
1500	12	19.4451	1.4441	18.2387	23.3923
2000	11	59.0628	3.7403	56.4010	69.3032
2500	11	118.9019	9.8519	94.6329	135.6236
3000	11	208.8783	12.9460	180.2307	237.0264
3500	11	309.7991	21.4969	270.4017	362.9151
4000	11	517.0497	32.1614	462.4882	596.3382
4500	10	743.6935	78.7802	598.3836	916.1887
5000	10	999.4138	120.6634	807.9537	1227.7434

V-L. *hilos8.2.doc*

Tamaño	n	Promedio (s)	Desv. est. (s)	Mínimo (s)	Máximo (s)
1000	12	0.9421	0.0572	0.8091	0.9974
1500	12	3.3714	0.1656	3.0887	3.7601
2000	12	7.8454	0.3858	6.9968	8.4807
2500	12	16.5511	0.8906	14.4802	17.9648
3000	5	29.7223	1.9966	26.6387	31.8346
3500	5	54.1236	13.8846	44.7436	78.7077
4000	5	82.1051	13.3124	69.2211	101.4737
4500	3	115.8866	19.5346	101.5143	138.1285
5000	3	150.7277	8.4517	141.0447	156.6232

VI. Gráficas

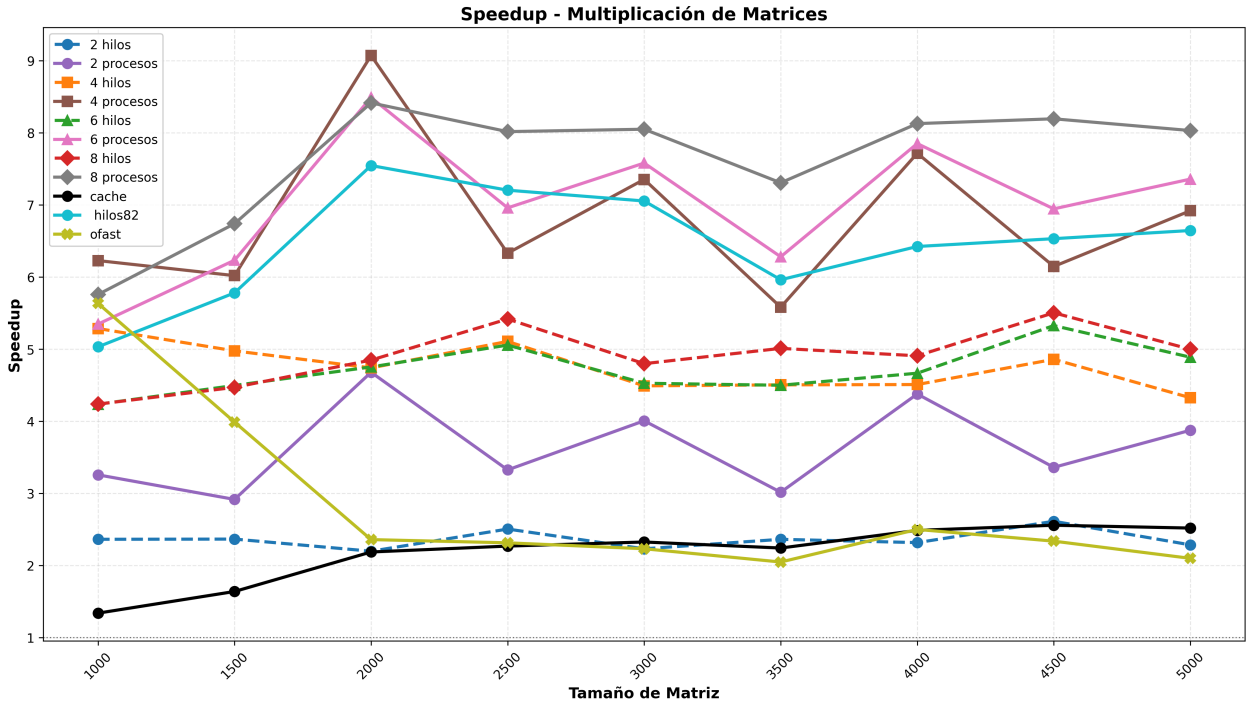

Figura 1: Speedup respecto a la versión estándar (standar.doc).

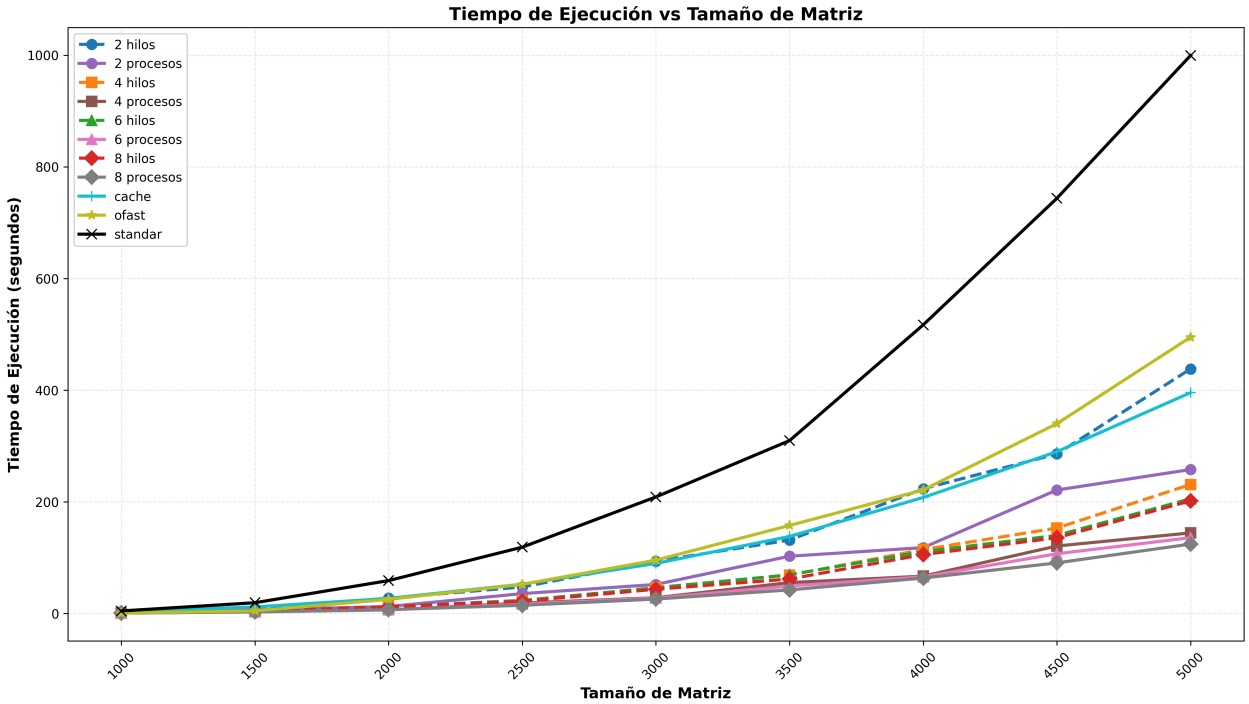

Figura 2: Tiempo de ejecución promedio vs tamaño de matriz.

VII. Conclusiones

1. **Impacto de la Paralelización:** Se demostró que la implementación de hilos y/o procesos en lenguaje C reduce significativamente el tiempo de procesamiento en matrices de gran escala (superiores a 2000x2000). Mientras que la ejecución secuencial presenta un crecimiento cúbico pronunciado, la versión paralela logra suavizar esta curva aprovechando los recursos multi-núcleo del procesador.
2. **Límite de Escalabilidad (Speedup):** Al analizar el *Speedup*, se observa que el beneficio de añadir hilos no es lineal de manera infinita. Existe un punto de eficiencia máxima cercano a los 4 hilos. Esto coincide con la arquitectura del hardware utilizado (4 núcleos físicos / 8 hilos lógicos), donde después de saturar los núcleos físicos, la ganancia por Hyper-Threading es marginal debido a la competencia por los recursos de caché y unidades de punto flotante.
3. **Impacto de la Estructura de Memoria en el Rendimiento:** Se concluye que la superioridad de la implementación basada en procesos no se debe únicamente al paralelismo, sino a la disposición de los datos en la memoria RAM. Mientras que la versión de hilos utilizó asignación dinámica fragmentada (*malloc* por cada fila), la versión de procesos implementó un bloque de memoria contigua mediante `shmget`. Esta estructura contigua maximiza el aprovechamiento de las líneas de caché del procesador i5-10300H y permite al *hardware prefetcher* anticipar las lecturas de datos, reduciendo drásticamente los tiempos de espera del CPU.
4. **Eficiencia de Procesos vs. Hilos en Arch Linux:** Experimentalmente se demostró que los procesos, al trabajar sobre un mapa de memoria virtual independiente pero con acceso a un segmento de memoria compartida (*IPC Shared Memory*), presentan una menor contención en el TLB (*Translation Lookaside Buffer*) que la implementación de hilos actual. Esto sugiere que para algoritmos de HPC con gran volumen de datos, evitar la fragmentación de memoria es tan crítico como la elección del modelo de ejecución.
5. **Eficiencia en la Planificación del Kernel:** Se concluye que el planificador de Arch Linux gestionó de manera más eficiente la distribución de procesos independientes sobre los núcleos del i5-10300H en comparación con los hilos. Esto se traduce en un *Speedup* superior, ya que cada proceso pudo mantener una localidad de datos más robusta en sus registros internos antes de escribir los resultados finales en la memoria compartida.

6. **Sinergia entre Software y Compilador:** Se determinó que las optimizaciones de código manual (como la memoria contigua) se potencian significativamente con el uso de la bandera `-Ofast`. La capacidad del compilador para vectorizar los bucles internos, sumada a una disposición de memoria eficiente, permitió alcanzar el techo de rendimiento teórico del hardware utilizado, siendo la mayor limitante que algunas instancias dejan de dar resultados coherentes con ciertas optimizaciones.

REFERENCIAS

[1] A. Cortéz, "Teoría de la complejidad computacional y teoría de sistemas," *Revista de Investigación de Sistemas e Informática*, vol. 1, no. 1, pp. 102-105, 2004. [En línea]. Disponible en: https://revistasinvestigacion.unmsm.edu.pe/index.php/sistem/article/view/3216

[2] A. López, "Introducción al High Performance Computing," *Encamina: Por una nube sostenible*, nov. 2017. [En línea]. Disponible en: https://blogs.encamina.com/por-una-nube-sostenible/introduccion-al-high-performance-computi

[3] K. Linux, "shmget(2) — Linux manual page," *man7.org*, 2023. [En línea]. Disponible en: https://man7.org/linux/man-pages/man2/shmget.2.html

[4] Free Software Foundation, "Options That Control Optimization: -Ofast," *Using the GNU Compiler Collection (GCC)*, 2024. [En línea]. Disponible en: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[5] IEEE Computer Society, "POSIX.1-2017: pthread.h - threads," *The Open Group Base Specifications Issue 7*, 2018.

[6] Arch Wiki Community, "Kernel: Optimization and Performance," *Arch Linux Wiki*, 2024. [En línea]. Disponible en: https://wiki.archlinux.org/title/Kernel